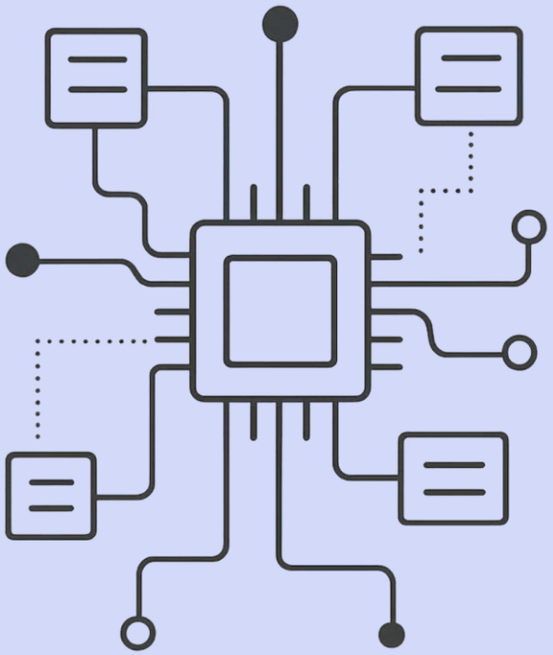


Punteros y Tipos Abstractos de Datos

UNIDAD 2

Una exploración profunda de los mecanismos fundamentales que permiten a los programas gestionar memoria, organizar información y escalar con eficiencia.

Contenidos

Punteros. Definición. Asignación estática y dinámica de memoria, aritmética de direcciones, punteros de punteros.
Tipo de Dato: Concepto. Tipos elementales de datos. Tipos de Datos Abstractos (TDA). Construcción de Tipos de Datos Abstractos. Estructuras de Datos. Ejemplos de estructuras de datos.

U2

El Poder de los Punteros

Los punteros son uno de los conceptos más poderosos y desafiantes de la programación de bajo nivel. Dominarlos significa entender cómo el software interactúa directamente con la memoria del sistema, abriendo la puerta a estructuras de datos dinámicas, gestión eficiente de recursos y algoritmos avanzados.

¿Qué es un Puntero? La Llave a la Memoria

Definición Técnica

Un puntero es una variable que almacena la **dirección de memoria** de otra variable, en lugar de almacenar un valor directo. Es como tener la dirección de una casa en lugar de la casa misma.

Permite acceder y manipular datos directamente en la memoria, sin copias innecesarias.

¿Por qué son fundamentales?

- Gestión dinámica de memoria en tiempo de ejecución
- Implementación de estructuras de datos complejas
- Paso eficiente de parámetros a funciones
- Acceso directo a hardware y sistemas embebidos

Gestión de Memoria: Estática vs. Dinámica

La forma en que se reserva la memoria determina la flexibilidad y el rendimiento de un programa. Los punteros son el puente entre el programador y la memoria dinámica.

Memoria Estática

Reservada en **tiempo de compilación**. El tamaño es fijo y conocido antes de ejecutar el programa.

- Variables locales y globales
- Ejemplo: `int x = 5;`
- Arreglos de tamaño fijo: `int arr[10];`
- Gestión automática por el compilador

Memoria Dinámica

Reservada en **tiempo de ejecución**. El tamaño puede ajustarse según las necesidades del programa.

- Operadores `new` y `delete` (C++)
- Funciones `malloc()` y `free()` (C)
- Estructuras de tamaño variable
- Requiere gestión manual para evitar fugas

Operaciones Clave con Punteros

Operador de Dirección (&)

Obtiene la dirección de memoria donde reside una variable.
Ejemplo: `p = &x;` almacena en `p` la dirección de `x`.

Operador de Indirección (*)

Accede al valor almacenado en la dirección a la que apunta el puntero.
Ejemplo: `y = *p;` copia el valor apuntado por `p` en `y`.

Asignación (=)

Copia la dirección de un puntero a otro, haciendo que ambos apunten a la misma ubicación de memoria. Ejemplo: `p2 = p1;`

Comparación (==, !=)

Determina si dos punteros apuntan a la misma dirección de memoria. Fundamental para controlar recorridos de listas y estructuras dinámicas.

Punteros de Punteros: Niveles de Abstracción

¿Qué es un puntero d‹e puntero?

Un puntero de puntero (`int **pp`) almacena la **dirección de otro puntero**, creando un segundo nivel de indirección. Es como tener un mapa que te dice dónde encontrar otro mapa.

🗋 Ejemplo:

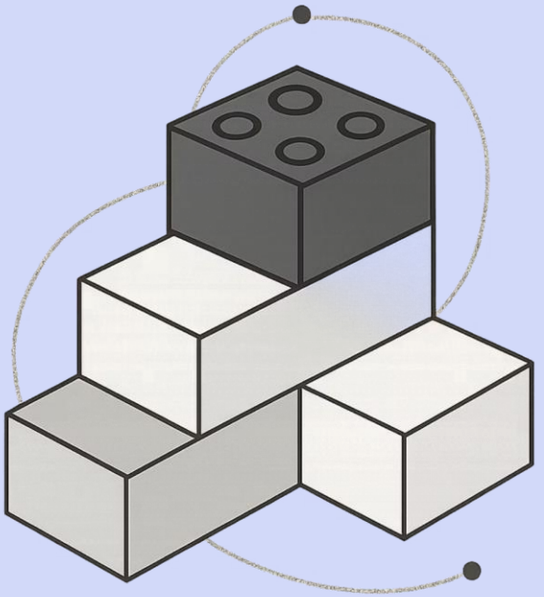
```
int x = 10;

int *p = &x;

int **pp = &p; // **pp devuelve el valor 10.
```

Aplicaciones Prácticas

→ **Arreglos dinámicos de punteros:** Matrices bidimensionales asignadas dinámicamente.

→ **Modificar punteros en funciones:** Pasar la dirección de un puntero para modificarlo desde otra función.

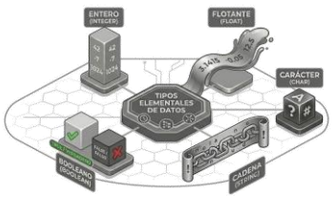
→ **Estructuras anidadas:** Árboles, grafos y estructuras jerárquicas complejas.

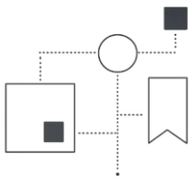


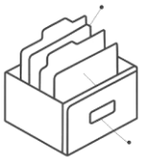
Tipos de Datos y Estructuras

Una vez dominados los punteros, el siguiente paso es comprender cómo organizar la información de manera eficiente.
Los tipos de datos y las estructuras son los cimientos sobre los que se construye todo software robusto y escalable.

Tipos de Datos: Los Bloques de Construcción







Tipos Elementales

Provistos directamente por el lenguaje de programación. Son la unidad mínima de información: **enteros** (int), **flotantes** (float, double), **caracteres** (char) y **booleanos** (bool). Ocupan un tamaño fijo en memoria.

Tipos de Datos Abstractos (TDA)

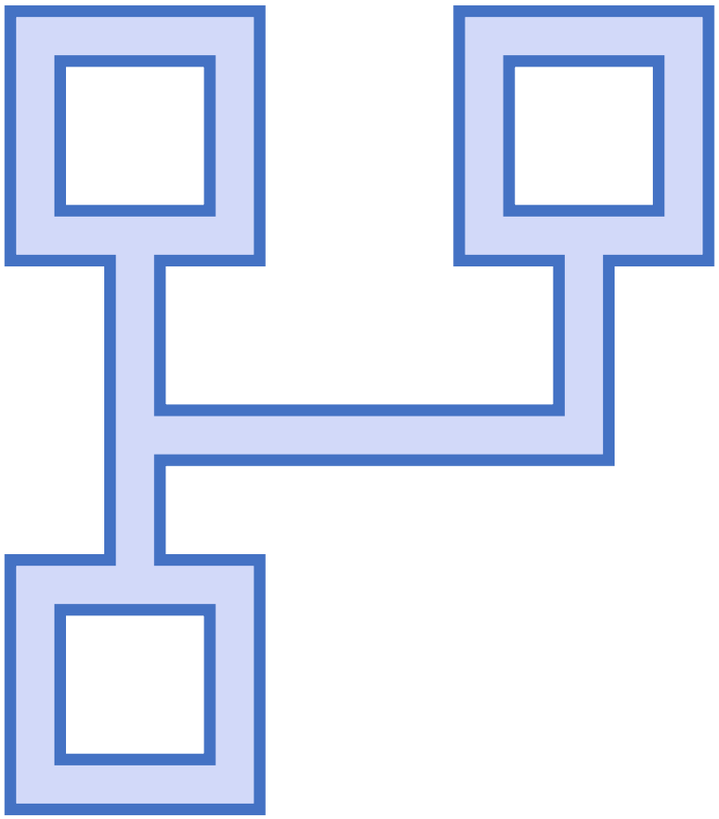
Conceptos definidos por el **problema a resolver**, independientes del lenguaje. Ejemplos clásicos: una **Pila** (LIFO), una **Cola** (FIFO), un **Conjunto**. Se especifican por sus operaciones, no por su implementación.

Estructuras de Datos

Implementaciones **concretas** de los TDAs, organizando los datos en memoria para un acceso y manipulación eficientes. Ejemplos: arreglos, listas enlazadas, árboles binarios, tablas hash y grafos.

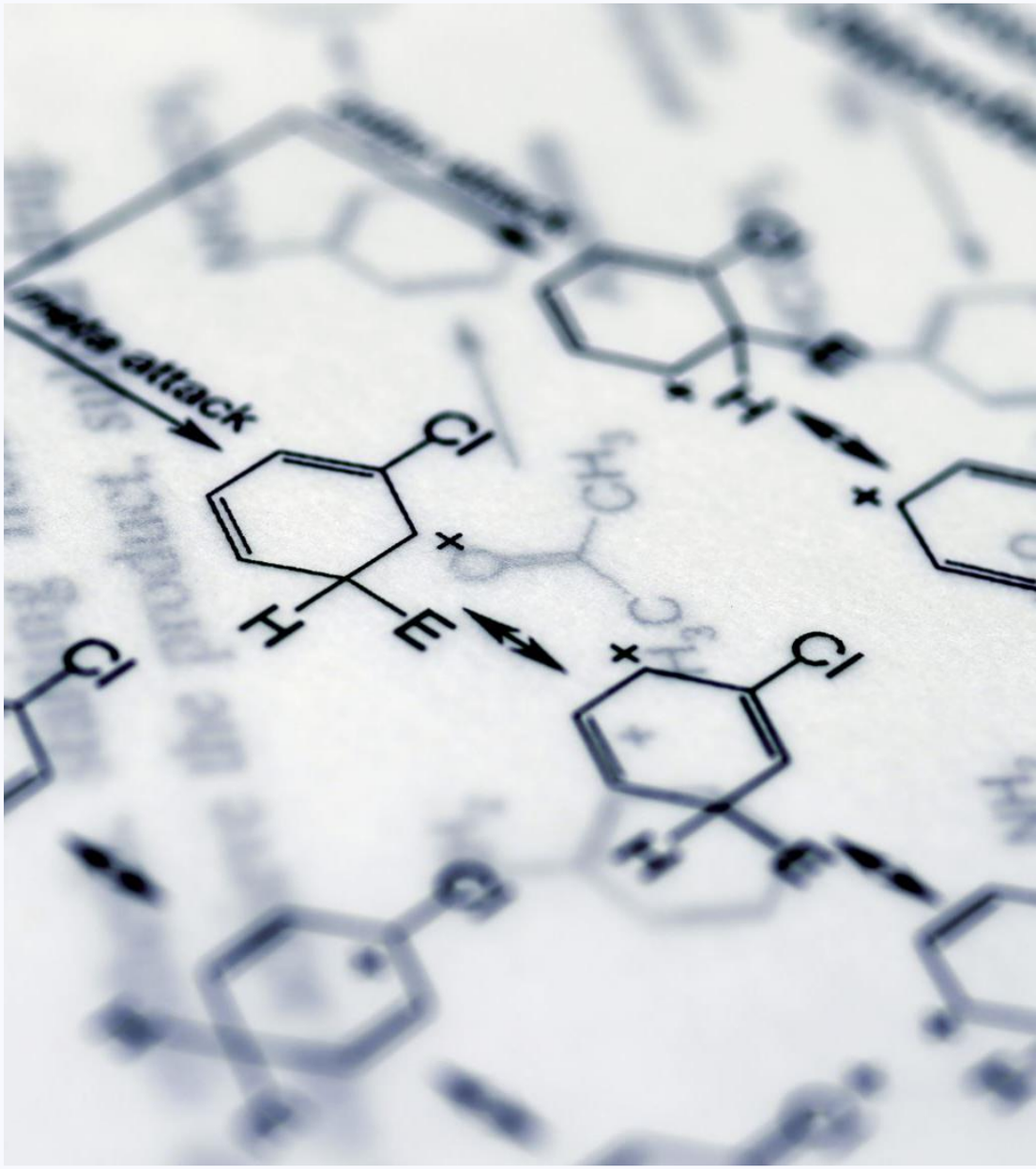
Tipos Abstractos de Datos

- La abstracción de datos es la técnica de programación que permite ***inventar*** o ***definir*** nuevos tipos de datos *(tipos de datos definidos por el usuario)* adecuados a la aplicación que se desea realizar.
- *Un tipo de datos definido por el programador se denomina "tipo abstracto de datos: TAD".*

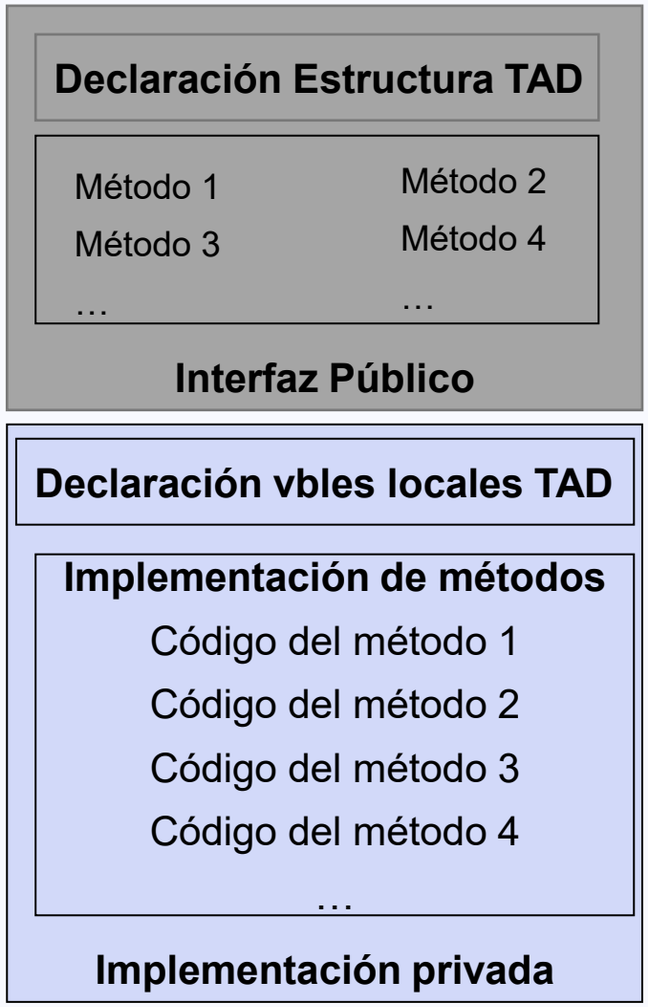


Tipos Abstractos de Datos

- Un TAD es un tipo de datos que consta de ***datos*** (estructura de datos propias) y ***operaciones*** que se pueden realizar sobre esos datos.
- Un TAD se compone de estructuras de datos y los procedimientos o funciones que manipulan esas estructuras de datos.
- Las operaciones desde un enfoque orientado a objetos se suelen denominar “métodos”. La estructura de un TAD (clase) desde un punto de vista global se compone de la ***interfaz*** y de la ***implementación***.



Tipos Abstractos de Datos



Características de un TAD

- ***ENCAPSULAMIENTO*:** Se desconoce la implementación de la *Declaración* y de las *Operaciones* del TAD.

- ***PROTECCIÓN*:** Sólo es posible acceder al TAD a través de las operaciones del mismo.

- Representa una ***ABSTRACCIÓN*:** Se seleccionan ciertos datos que interesan del mundo real, ignorando el resto.

- Deben poder ***COMPILARSE POR SEPARADO***.

Características de un TAD

- **Especificación del TAD:** Permite al usuario la utilización del TAD.

- **Implementación del TAD:** Sólo la conoce el programador del TAD, no el usuario.

- **Especificaciones**
  - ***Sintácticas:*** Hacen referencia a aspectos Sintácticos: Nombre de la operación, Parámetros de entrada, Parámetros de salida, Tipo de dichos parámetros, Resultado devuelto por la operación, Tipo del resultado, etc.
  - ***Semánticas:*** Indican el efecto producido por la operación.

Para CADA OPERACIÓN del TAD es necesario dar sus especificaciones ***Sintácticas*** y ***Semánticas***.

TAD: Ejemplo

- En C++ el tipo ***Punto***, que representa a las coordenadas ***x*** e ***y*** de un sistema de coordenadas rectangulares, ***no existe***.
- Sin embargo, es posible implementar el tipo abstracto de datos, considerando los ***valores*** que se almacenan en las variables y qué ***operaciones*** están disponibles para manipular estas variables.

TAD: Ventajas

- Modularidad
- Adecuados para sistemas no triviales
- Separación entre ***especificación*** e ***implementación***. Esto hace al sistema:
  - más legible
  - más fácil de mantener
  - más fácil de verificar y probar que es correcto. Robustez.
  - más fácil de reusar
  - más extensible
  - lo independiza en cierta manera de las distintas implementaciones (complejidad tiempo-espacio).
- La genericidad es una característica deseable de un TAD, ya que fomenta el reuso.
- El uso de TADs permite una rápida prototipación de sistemas, dejando de lado detalles de eficiencia para una etapa posterior.

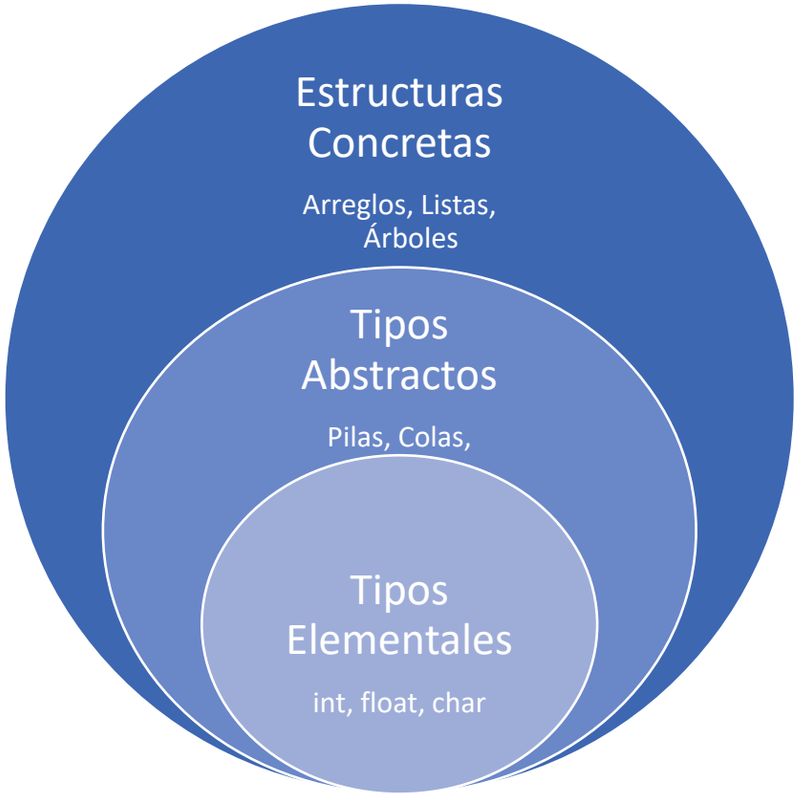
Construyendo TDAs y Estructuras de Datos

Los punteros son el ingrediente esencial para construir estructuras de datos dinámicas y flexibles. Cada estructura tiene ventajas específicas según el problema a resolver.

Arreglos
Colecciones de elementos del **mismo tipo** en memoria contigua. Acceso directo por índice en O(1). Tamaño fijo en asignación estática.

Listas Enlazadas
Cada **nodo** contiene datos y un puntero al siguiente elemento. Inserciones y eliminaciones eficientes sin reordenar memoria. Tamaño dinámico.

Árboles y Grafos
Estructuras jerárquicas donde cada nodo contiene **punteros a múltiples nodos** hijos. Base de búsquedas eficientes, redes y sistemas de archivos.



Esta jerarquía muestra cómo los tipos elementales se combinan para formar TDAs conceptuales, que a su vez se implementan como estructuras de datos concretas utilizando punteros y gestión dinámica de memoria.

El Futuro: Flexibilidad y Eficiencia

La combinación de punteros, gestión de memoria dinámica y Tipos de Datos Abstractos permite crear software **robusto, escalable y eficiente**. Estos conceptos son la base invisible de prácticamente todo el software moderno.

∞

Estructuras Dinámicas

Sin límite de tamaño predefinido gracias a la memoria dinámica.

O(1)

Acceso Directo

Los arreglos permiten acceso instantáneo a cualquier elemento por índice.

3

Niveles de Abstracción

Elemental → TDA → Estructura concreta: el camino del diseño eficiente.

Conclusión clave: Desde sistemas operativos hasta aplicaciones web, desde motores de videojuegos hasta bases de datos, los punteros y las estructuras de datos son los cimientos sobre los que se construye toda la programación moderna.

AUTO (Ejemplo)

Atributos

(Lo que el auto "es")

- **Marca y Modelo**: Identificadores del vehículo.
- **Nivel de Combustible**: Cantidad actual en el tanque (0 a 100%).
- **Estado del Motor**: Si está encendido o apagado.
- **Velocidad Actual**: Rapidez a la que se desplaza.

Operaciones

(Lo que el auto "hace")

- **Encender/Apagar**: Cambia el estado del motor.
- **Acelerar:** Aumenta la velocidad (siempre que haya combustible y el motor esté encendido).
- **Frenar:** Disminuye la velocidad hasta detenerse.
- **Repostar:** Carga combustible al tanque.
- **Consultar Estado:** Ver la velocidad o combustible actual.

Auto: Definición en C++

```
#ifndef AUTO_HPP
#define AUTO_HPP

#include <string>

// Definición de la estructura de datos
struct Auto {
    char marca[50];
    char modelo[50];
    double nivelCombustible;
    double velocidad;
    bool motorEncendido;
};
```

```
// --- Operaciones (Interfaz del TAD) ---

// Inicialización
void inicializarAuto(Auto &a, const char* marca, const
char* modelo);

// Acciones
void encenderMotor(Auto &a);
void apagarMotor(Auto &a);
void acelerar(Auto &a, double cantidad);
void frenar(Auto &a, double cantidad);
void repostar(Auto &a, double cantidad);

// Consultas (Usamos const para indicar que no
modificamos el auto)
double obtenerVelocidad(const Auto &a);
bool estaEncendido(const Auto &a);

#endif
```

THE END!